

LOSS FUNCTIONS

- Machines learn by means of a loss function.
- It's a method of evaluating how well specific algorithm models the given data.
- If predictions deviates too much from actual results, loss function would give a very large number
- Gradually, with the help of some optimization function, loss function learns to reduce the error in prediction

REMEMBER:

The objective is to minimize the loss between the predictions and the actual outputs

CLASSIFICATION OF LOSS FUNCTIONS

- Loss functions can be classified into two major categories depending upon the type of learning task we are dealing with —
 1. **Regression losses**
 2. **Classification losses**
- In classification, we are trying to predict output from set of finite categorical values
- Regression, on the other hand, deals with predicting a continuous value for example given floor area, number of rooms, size of rooms, predict the price of room.

REGRESSION LOSSES

- Regression, deals with predicting a continuous value for example given floor area, number of rooms, size of rooms, predict the price of room.
- Regression losses are:
 - ✓ Mean Squared Error (L2 loss)
 - ✓ Mean Absolute Error (L1 loss)
 - ✓ Huber Loss

Mean Squared Error(L2 loss or Quadratic loss)

- *Mean square error* is measured as the average of squared difference between predictions and actual observations.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y - y_i)^2$$

n=Total number of observation

y_i = predicted value

y = actual value.

Variations of Mean Squared Error

- Half of the Mean Squared Error

$$MSE = \frac{1}{2n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2$$

- Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2}$$

Example of MSE

Sample	Predicted	Actual	Error	Squared Error
1	48	60	-12	144
2	51	53	-2	4
3	57	60	-3	9

$$\frac{1}{3} \sum_{i=1}^3 144 + 4 + 9 = \frac{157}{3} = 52.3$$


**BIG!!!
MINIMIZE**

Mean Absolute Error(L1 Loss)

- *Mean absolute error(MAE)*, on the other hand, is measured as the average of sum of absolute differences between predictions and actual observations.

$$MAE = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{n}$$

L1 or L2 LOSS

- L2 loss is more sensitive to outliers than L1 loss
- When there are large deviations, the error is big, and when squaring a big number, it gets bigger

Example: If error is 10, then MEA would give 10 and MSE would give 100

- Consequently L1 loss function is more robust and is generally not affected by outliers
- On the other hand L2 loss function will try to adjust the model according to these outliers values even on the expense of the other samples
- Hence L2 loss function is highly sensitive to outliers in the dataset

HUBER LOSS(SMOOTH L1 LOSS)

- The Huber loss combines the best properties of MSE and MAE.
- It is quadratic for smaller errors and is linear otherwise (and similarly for its gradient).
- It is identified by its *delta* parameter:

$$\text{Huber} = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (y_i - \hat{y}_i)^2 \quad |y_i - \hat{y}_i| \leq \delta$$
$$\text{Huber} = \frac{1}{n} \sum_{i=1}^n \delta \left(|y_i - \hat{y}_i| - \frac{1}{2} \delta \right) \quad |y_i - \hat{y}_i| > \delta$$

CLASSIFICATION LOSSES

- Classification losses are:
 - ✓ Binary cross entropy
 - ✓ Categorical cross entropy
 - ✓ Hinge Loss

BINARY CROSS ENTROPY LOSS

- Usually used when output labels have 0 or 1
- Also used when output labels have values between 0 and 1
- It is widely used when we have two classes (0 or 1, yes or no)

$$BCE\ Loss = -\frac{1}{n} \sum_{i=1}^n [y \log(p) + (1 - y) \log(1 - p)]$$

- This is also called **Log-Loss**.
- To calculate the probability p , we can use the sigmoid function.

Notice that when actual label is 1 ($y = 1$), second half of function disappears whereas in case actual label is 0 ($y = 0$) first half is dropped off.

CATEGORICAL CROSS ENTROPY

- Categorical Cross Entropy loss is essentially Binary Cross Entropy Loss expanded to multiple classes.
- Binary cross-entropy is for binary classification and categorical cross
- Categorical cross-entropy is based on the assumption that only 1 class is correct out of all possible ones

CATEGORICAL CROSS ENTROPY

- *The target for multi-class classification is a one-hot vector, meaning it has 1 on a single position and 0's everywhere else.*
- If our target is a one-hot vector, we can indeed forget targets and predictions for all the other classes and compute only the loss for the hot class.

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

- This is called **categorical cross-entropy** — a special case of cross-entropy, where our target is a one-hot vector.

HINGE LOSS

- Hinge loss is primarily used with Support Vector Machine (SVM) Classifiers with class labels -1 and 1
- Hinge Loss not only penalizes the wrong predictions but also the right predictions that are not confident.

$$\text{For all samples: } \sum \max(0, 1 - y * \hat{y})$$

Where y is the actual label (-1 or 1) and $\hat{y}$ is the prediction.

The loss is 0 when the signs of the label and prediction match.

$$\text{For all samples: } \sum \max(0, 1 - y * \hat{y})$$

Consider the prediction when the actual label is -1:

$\max[0, 1 - (-1 * 0.3)] = \max[0, 1.3] = 1.3 \rightarrow$ Loss is high

$\max[0, 1 - (-1 * -0.8)] = \max[0, 0.2] = 0.2 \rightarrow$ Loss is low

$\max[0, 1 - (-1 * -1.1)] = \max[0, -0.1] = 0 \rightarrow$ No Loss!

$\max[0, 1 - (-1 * -1)] = \max[0, 0] = 0 \rightarrow$ No Loss!

$\max[0, 1 - (-1 * 1)] = \max[0, 2] = 0 \rightarrow$ Loss is very high

Calculate the huber loss for the following data if $\delta=0.5$

actual values of y	4	-1.5	5	2
predicted values of $\hat{y}$	3.5	1	5	3

$$\delta = 0.5$$

$$\begin{aligned} \text{Huber} &= \frac{\frac{(4 - 3.5)^2}{2} + 0.5 \cdot \left(|-1.5 - 1| - \frac{1}{2} \cdot 0.5\right) + \frac{(5 - 5)^2}{2} + 0.5 \cdot \left(|2 - 3| - \frac{1}{2} \cdot 0.5\right)}{4} = \\ &= \frac{0.125 + 1.125 + 0 + 0.375}{4} \approx 0.406 \end{aligned}$$

$$\begin{aligned} \text{Huber} &= \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (y_i - \hat{y}_i)^2 & |y_i - \hat{y}_i| \leq \delta \\ \text{Huber} &= \frac{1}{n} \sum_{i=1}^n \delta \left(|y_i - \hat{y}_i| - \frac{1}{2} \delta \right) & |y_i - \hat{y}_i| > \delta \end{aligned}$$

GRADIENT DESCENT

- Gradient descent is an optimization algorithm which is commonly-used to train machine learning models and neural networks.
- It is the base algorithm that is used to minimize the error with respect to the weights of a neural network
- Gradient descent is an optimization algorithm that works iteratively to find the model parameters with minimal cost or error values.
- Until the function is close to or equal to zero, the model will continue to adjust its parameters to yield the smallest possible error.

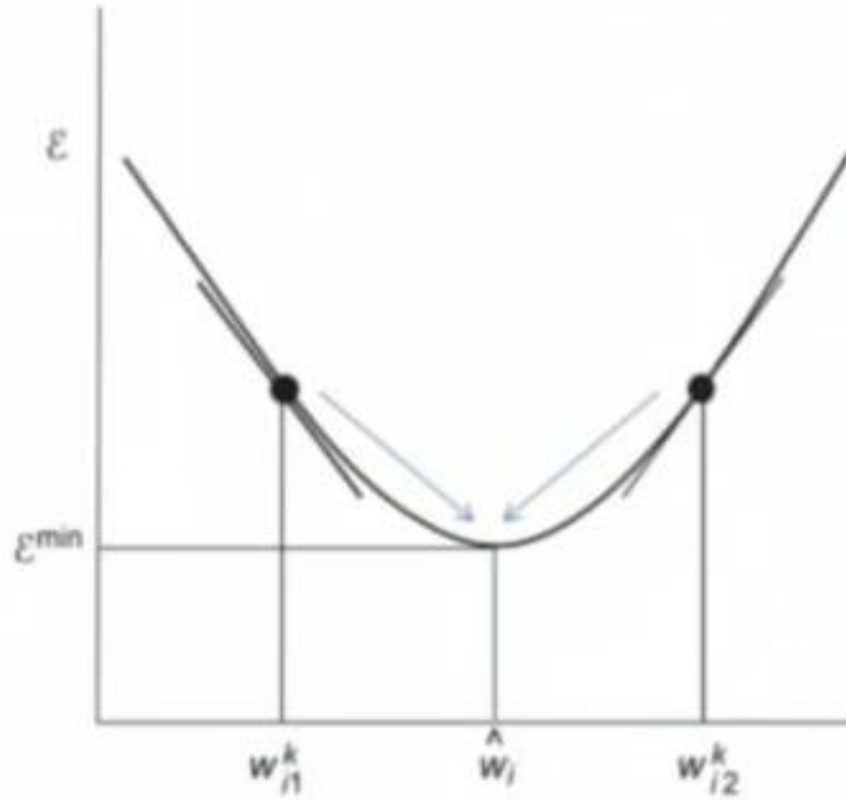
Gradient Descent

Negative Slope:

When we **Increase** w , the loss function is **decreasing** $\rightarrow$
 $-(-) = + \rightarrow$ Weight Increases (Moving Right)

Positive Slope:

When we **increase** w , the loss function is **increasing**.
 $-(+) = - \rightarrow$ Weight Decreases (Moving Left)



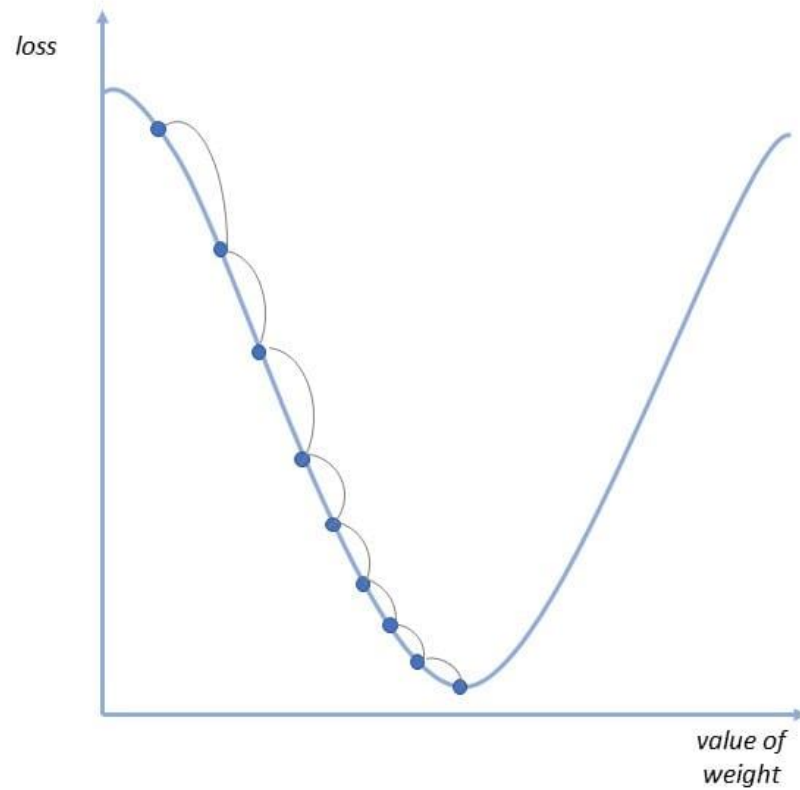
$$w_{\text{new}} = w - \alpha \frac{\partial L}{\partial w}$$

Learning rate(step size)

- **Learning rate** (also referred to as step size or the alpha) is the size of the steps that are taken to reach the minimum.
- This is typically a small value, and it is evaluated and updated based on the behavior of the cost function.
- High learning rates result in larger steps but risks overshooting the minimum.
- Conversely, a low learning rate has small step sizes. While it has the advantage of more precision, the number of iterations compromises overall efficiency as this takes more time and computations to reach the minimum.

Learning rate - how fast we update the weights

Small Learning Rate



Large Learning Rate

